



Chapter 1

Introduction to
Embedded Systems

1.1 Roadmap

1.1.1 What You Will Learn

After you have studied the material in this chapter, you will be able to:

- Implement basic programs on the NUCLEO board using flow control, logical operators, and variables.
- Explain the basic concepts about how microcontrollers operate.

1.1.2 Contents of This Chapter

In every embedded system there is an element that reads the inputs, drives the outputs, and controls the communications and interfaces. This element is usually a microcontroller. To perform its tasks, the microcontroller runs a program. In this chapter, the process of writing a program is explained as well as how to upload it to the NUCLEO-F429ZI board (hereinafter called the NUCLEO board) that is used in this book. Detailed information about this board can be found in [1].

The chapter also explains how to control the LEDs of the NUCLEO board and the connected buttons. These buttons are used in example problems related to a smart home system project.

The examples in this chapter make use of some elements of the Arm Mbed OS 6, which is a free, open-source, rapid development platform designed to help developers get started building *Internet of Things* (IoT) applications quickly. An introduction to Mbed OS 6 can be found in [2].

In this way, some of the main concepts about embedded programming using the C and C++ languages are introduced, such as *nested if* statements, OR and AND logical operators, and *Boolean* and *non-Boolean* variables.

Throughout the chapter, the reader is guided on how to *compile* the examples and load them onto the NUCLEO board using Keil Studio Cloud, which allows the developer to write code from scratch or import an existing project and modify it to suit the developer's requirements. An introduction to Keil Studio Cloud is available in [3].

To use Keil Studio Cloud, the only thing the reader needs is an Arm Mbed account. [4] explains how to create an Arm Mbed account or log in if an account is already created. Keil Studio Cloud is available from [5] and a quick-start guide is available from [6].



NOTE: Those readers who prefer to use a dedicated desktop setup can use the Mbed Studio IDE (*Integrated Development Environment*), which is available from [7], or any other IDE, for example the STM32CubeIDE, which is introduced in [8].

In the Under the Hood section, the basic principles of microcontrollers are introduced. Finally, in the Case Study section, a smart door lock based in Mbed OS is analyzed in order to show that many of the concepts introduced in this chapter are used in commercial products.

1.2 Fundamentals of Embedded Systems

1.2.1 Main Components of Embedded Systems

Throughout the chapters in this book, a smart home system project will be implemented. In this section, different smart home system implementations are analyzed in order to highlight the fundamentals of embedded systems.

Figure 1.1 shows four different implementations of a smart home system, adapted from different sources. The inputs are indicated in yellow, the microcontroller in light blue, the outputs in light red, the communication links in light gray, the human interfaces in blue, and the power supply in green. The elements contained in the block diagrams illustrated in Figure 1.1 are summarized in Table 1.1.

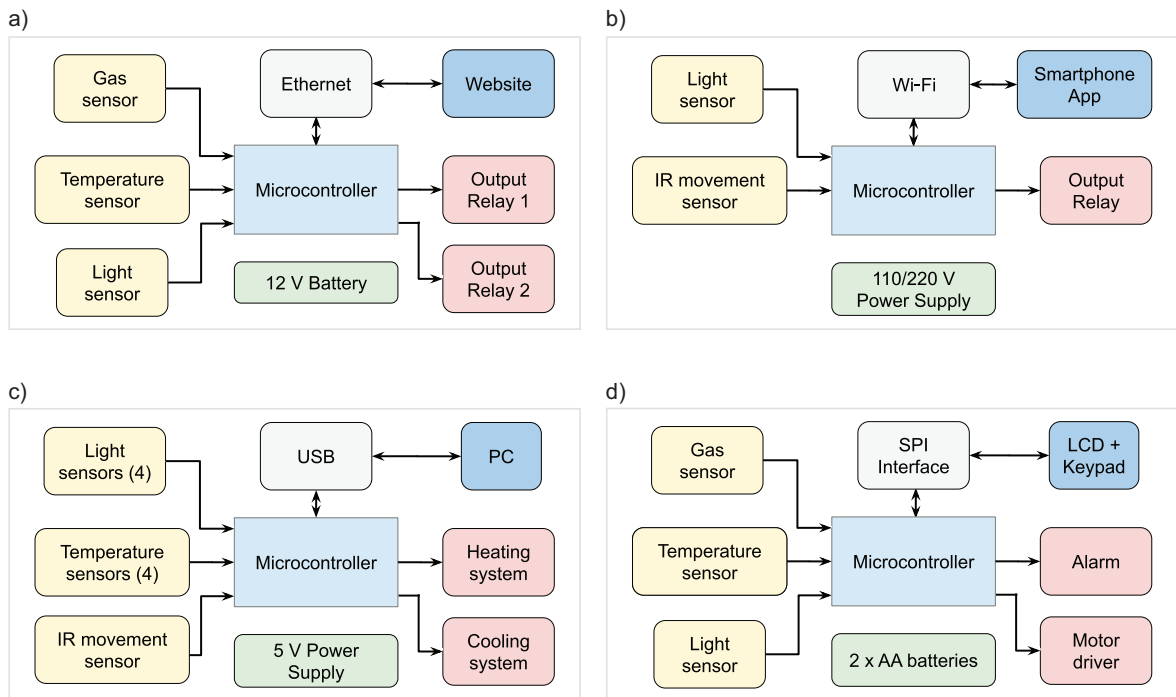


Figure 1.1 Four different block diagram representations of a smart home system.

Table 1.1 Elements contained in the smart home system block diagrams illustrated in Figure 1.1.

Smart Home	Inputs (yellow)	Microcontroller (light blue)	Outputs (light red)	Communication (light gray)	Interface (blue)	Power supply (green)
(a)	Gas, light, and temperature sensors	Yes	Two relays	Ethernet	Website	12 V battery
(b)	IR movement and light sensors	Yes	One relay	Wi-Fi	Smartphone App	110/220 V
(c)	Light, IR movement, and temperature sensors	Yes	Heating/cooling system	USB	PC	5 V
(d)	Gas, light, and temperature sensors	Yes	Alarm/motor driver	SPI	Keypad + LCD display	2 × AA batteries

From Table 1.1, the following conclusions can be made:

- All the implementations have a *microcontroller*.
- The type and number of sensors depends on the implementation.
- All the implementations have outputs, a relay being the most frequent one.
- Communications are implemented by different technologies (Ethernet, Wi-Fi, USB).
- Different interfaces are used: a website, a smartphone app, a keypad plus an LCD display.
- The power supply is indicated in the four diagrams (12 V battery, 110/220 V, etc.).

The microcontroller is the heart of the system: all the inputs are connected to the microcontroller and all the outputs are driven by the microcontroller, while the communications and the interfaces are controlled by the microcontroller. For this reason, every embedded system has one microcontroller (at least), or a similar element that plays the same role.



DEFINITION: A typical definition of an *embedded system* is a computer system that has a dedicated function within a larger mechanical or electrical system and is embedded as part of a complete device, often including electrical or electronic hardware and mechanical parts.

DEFINITION: A typical definition of a *microcontroller* is a small computer on a single integrated circuit (IC) chip which typically contains one or more processor cores along with memory and programmable peripherals.



NOTE: The Under the Hood section, at the end of this chapter, discusses in detail what is inside a microcontroller. The aim is to make the reader use a microcontroller before explaining how they work.

The microcontroller runs a program made up of instructions by means of which the programmer instructs the microcontroller what to do. This book aims to enable the reader to write this type of program and to connect the appropriate sensors, actuators, and interfaces to the inputs and outputs of the microcontroller. The sensors will provide measurements of the environment, while the actuators will allow the microcontroller to have control over elements and devices within the environment.

Proposed Exercises

1. Using Table 1.1 as a reference, propose a smart home system having a maximum of two types of sensors (inputs), one type of actuator (output), one communication link, and two human interfaces.
2. Make the block diagram of the proposed smart home system.

Answers to the Exercises

1. The smart home system may include a gas sensor, a temperature sensor, an alarm, a keypad, and a communication link with a PC.
2. The diagram of the proposed smart home system is shown in Figure 1.2.

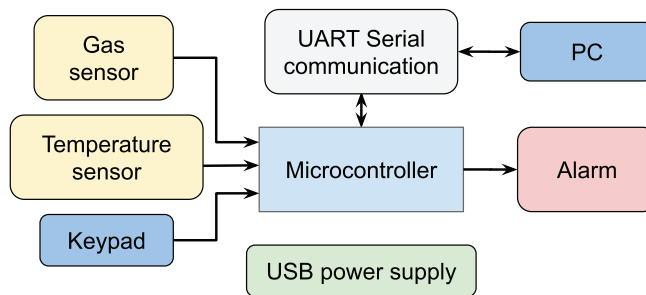


Figure 1.2 Diagram of the proposed smart home system.



NOTE: In Figure 1.2, the link with the PC is implemented by means of a UART serial communication (*Universal Asynchronous Receiver-Transmitter*). The UART peripheral is introduced later in Chapter 2.

1.2.2 First Implementation of the Smart Home System

This section will begin the implementation of the smart home system, following the diagram shown in Figure 1.3. This first implementation will detect fire using an over temperature detector and a gas detector and, if it detects fire, it will activate the alarm until a given code is entered. For this purpose, the NUCLEO board [1] provided with the STM32F429ZIT6U microcontroller [9] (referred to as the *STM32 microcontroller*) is used.

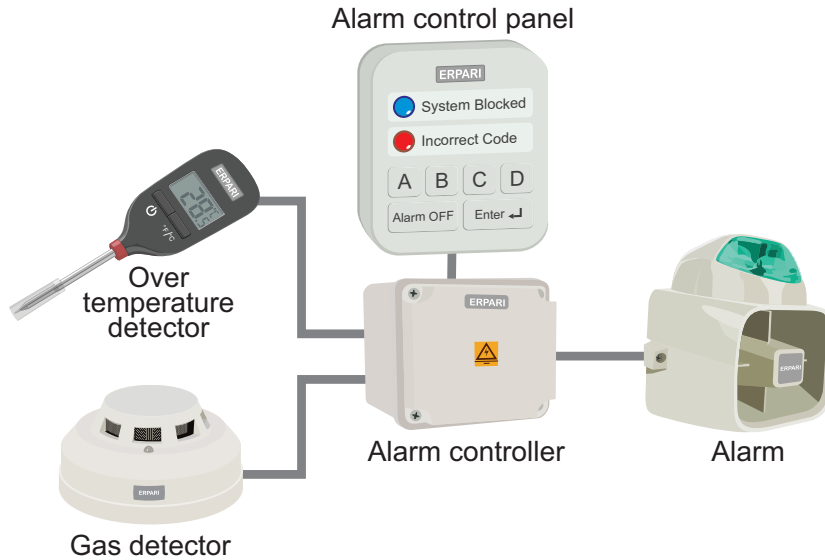


Figure 1.3 The smart home system that is implemented in this chapter.

The reader might notice that there are not as many buttons in the NUCLEO board as in the alarm control panel shown in Figure 1.3. Instead, the NUCLEO board is provided with only one user button and three LEDs, as shown in Figure 1.4. It can be concluded that a way to connect more buttons to the NUCLEO board is needed. For this purpose, the ST Zio connectors of the NUCLEO board shown in Figure 1.4 are used.

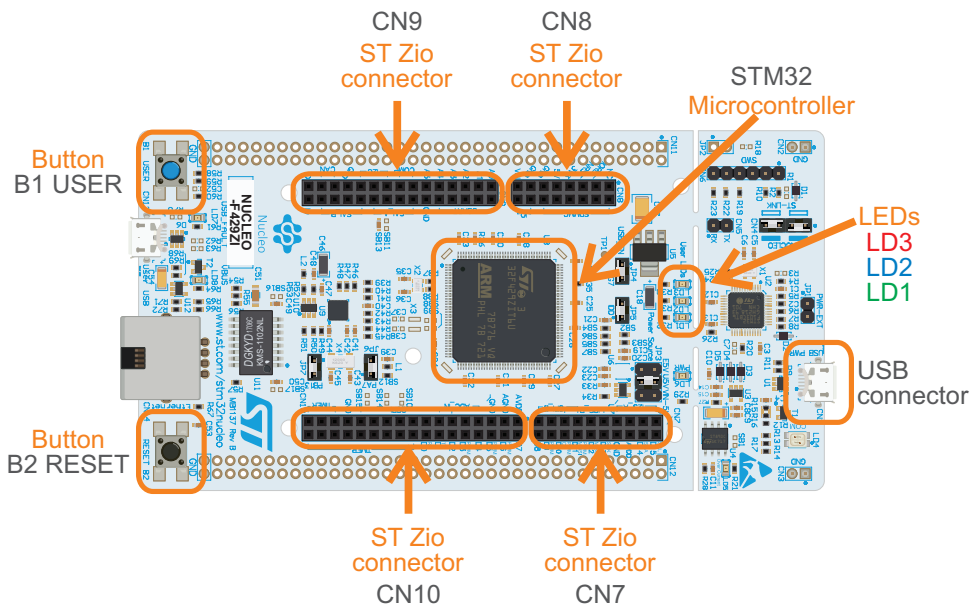


Figure 1.4 The NUCLEO-F429ZI board used in this book.

The overall diagram of the embedded system that will be connected in this chapter is shown in Figure 1.5. It can be seen that a breadboard is used, together with the most common buttons used in electronics, technically known as *tactile switches*. For breadboarding, any appropriate wire can be used, but it is recommended to use *jumper wires*.

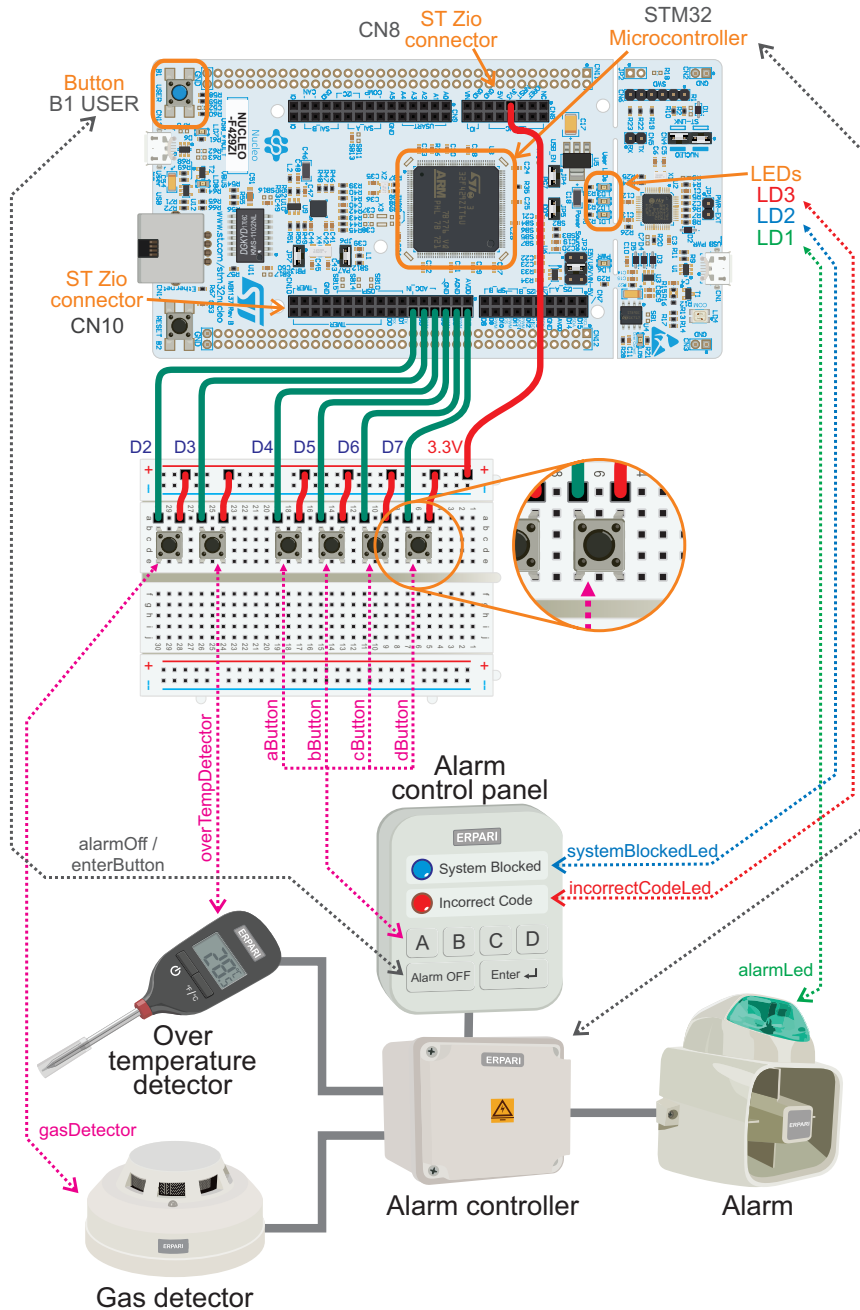


Figure 1.5 The elements of the smart home system introduced in Figure 1.3 with their corresponding components.

Figure 1.5 shows the elements of the smart home system introduced in Figure 1.3 and their proposed representation by means of the components connected. The names that will be used in the code to refer to each element are indicated in Figure 1.5 (i.e., gasDetector, alarmLed, etc.). It can be seen that the activation of the gas detector and the over temperature detector are simulated by buttons D2 and D3 placed over the breadboard. All these connections are summarized in Table 1.2.



TIP: It may be useful to indicate the function of each button and LED using a card next to each one.



NOTE: In Chapters 1 and 2, the alarm will be represented by an LED. In Chapter 3, a buzzer will be incorporated into the alarm in order to represent a siren. In later chapters it will be explained how to incorporate a strobe light into the alarm. In this way the alarm will have light and sound, as in Figure 1.5.

Table 1.2. Elements of the smart home system and the corresponding representation implemented.

Smart home system	Representation used
Alarm	LD1 LED (green)
System blocked LED	LD2 LED (blue)
Incorrect code LED	LD3 LED (red)
Alarm Off / Enter button	B1 USER button
Gas detector	Button connected to D2 pin
Over temperature detector	Button connected to D3 pin
A button	Button connected to D4 pin
B button	Button connected to D5 pin
C button	Button connected to D6 pin
D button	Button connected to D7 pin

In Table 1.2 it should be noticed that the B1 USER button is used for the Alarm Off and also for the Enter button. The specific use will vary during the examples and will be clearly indicated as it becomes necessary.

In Figure 1.6 and Figure 1.7 the details of the connections that should be made on the CN8 and CN10 ST Zio Connectors are shown.

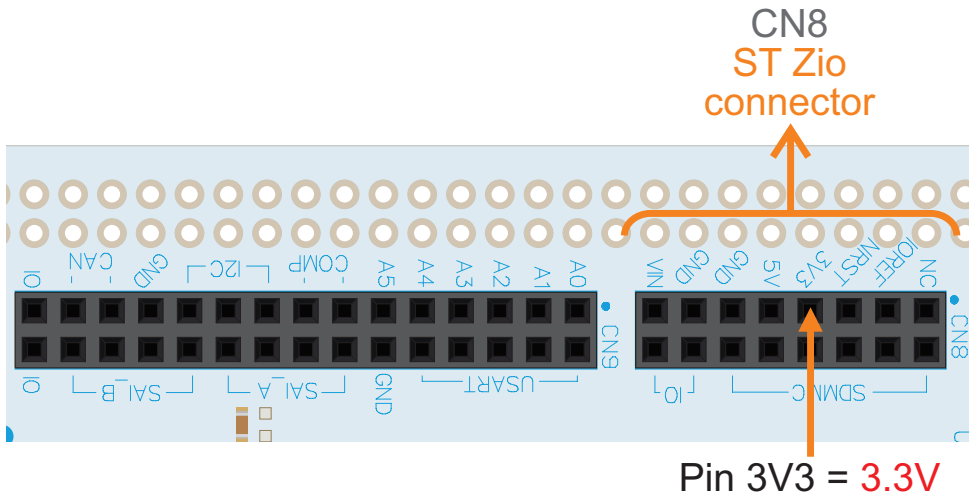


Figure 1.6 Detail of the connection made in the CN8 ST Zio connector of the NUCLEO board.

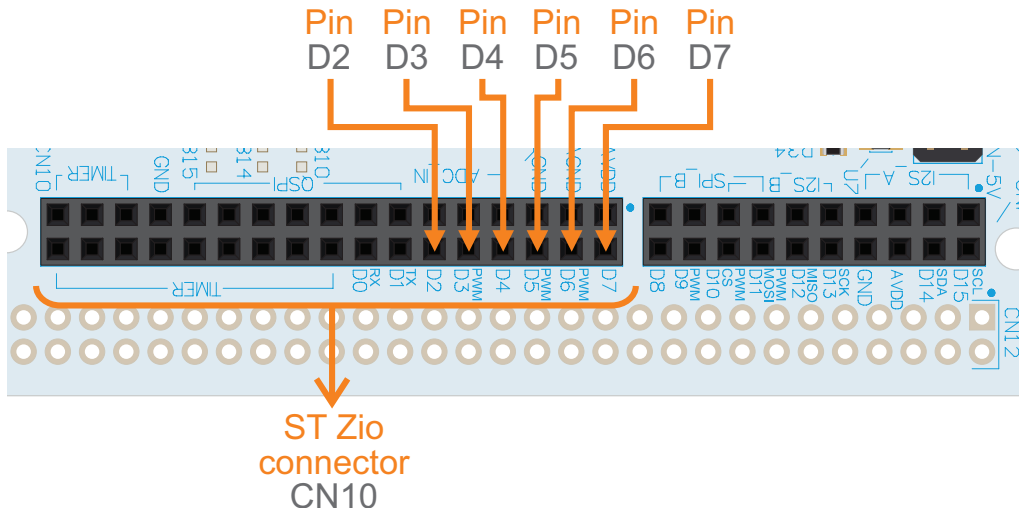


Figure 1.7 Detail of the connections made in the CN10 ST Zio connector of the NUCLEO board.



WARNING: It is crucial to connect the jumper wires and buttons exactly as indicated in Figure 1.5, Figure 1.6, and Figure 1.7. Otherwise, the programs presented in this chapter will not work.

To make all the connections, unplug the USB power supply from the NUCLEO board. Prior to reconnecting the USB power supply, check that there are no short circuits.

1.2.3 Getting Ready to Program the First Implementation of the Smart Home System

This subsection explains how to load a program onto the STM32 microcontroller of the NUCLEO board using Keil Studio Cloud in order to test if the buttons that were connected are working properly.

First, the reader must copy the URL of the repository of the “Subsection 1.2.3” program that is available in [10]. Then, in the “File” menu of Keil Studio Cloud, select “Import project”. The “Import project” window will be displayed in the web browser. Press “Add project” without modifying any of the configurations, and a new project named “Subsection 1.2.3” will be added to the list of available projects as the active project.



NOTE: If in the future changes are made in Keil Studio Cloud that alter the steps to compile and download the programs to the NUCLEO board, the corresponding instructions will be published in [10].

The program that was imported has the behaviors detailed in Table 1.3. When the B1 USER button of the NUCLEO board is pressed, LEDs LD1, LD2, and LD3 of the board are turned on. This is to ensure that the three LEDs and the program are working correctly. If the external buttons connected to D2 or D3 are pressed, LD1 is turned on. In this way it can be tested whether the buttons connected to D2 and D3 are properly connected and working. The same applies to the buttons connected to D4 and D5 by means of LD2, and the buttons connected to D6 and D7 by means of LD3.

Table 1.3 Behaviors of the program that is used in this subsection to test the buttons.

LED	Turns on if any of the following is pressed
LD1 (green)	B1 USER button, button connected to D2, button connected to D3
LD2 (blue)	B1 USER button, button connected to D4, button connected to D5
LD3 (red)	B1 USER button, button connected to D6, button connected to D7



NOTE: The code of the program that has been loaded onto the NUCLEO board to test if the six external buttons are working correctly will not be analyzed in this subsection. This is due to the fact that the behavior of this code is based on some elements that will be introduced and explained in the following subsections. For now, it is enough to be able to check if the six buttons are working.

In order to load the program onto the NUCLEO board, connect it to the computer as shown in Figure 1.8. If “NUCLEO-F429ZI” is not shown in the “Target hardware” section, press the “Find target” button. If a popup menu is displayed, select the “STM32 STLink” connection and follow the instructions. Once “NUCLEO-F429ZI” is shown in the “Target hardware” section, press the “Build project” button. The progress will be shown in the “Output” tab. Once the program has been compiled, a .bin file will be automatically downloaded by the web browser. Drag the downloaded .bin file to the drive assigned to the NUCLEO board (for example, D:\, named NODE_F429ZI). A “copying” window

will open, LED LD4 of the NUCLEO board will alternate red and green light, and when the window closes the program will already be running on the NUCLEO board.

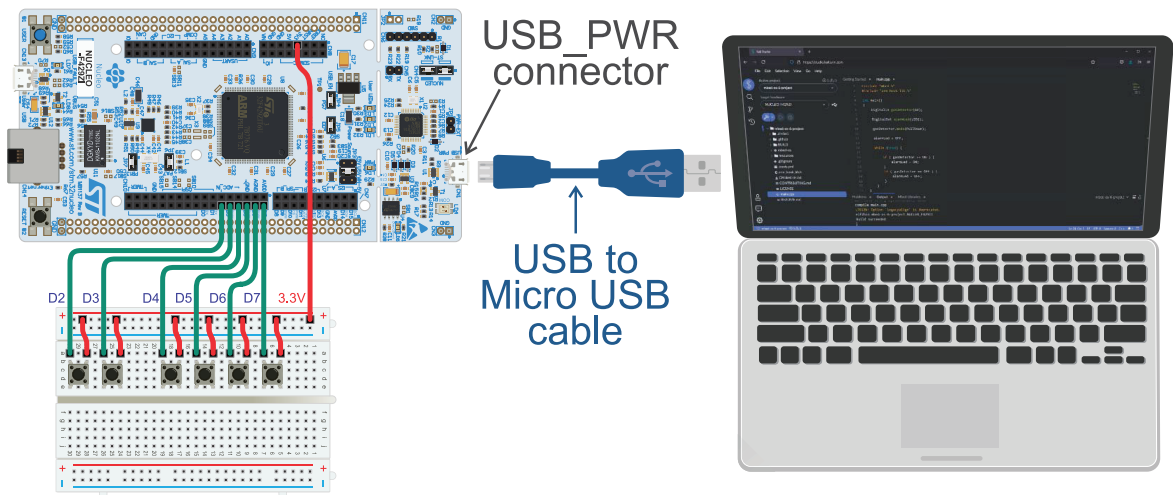


Figure 1.8 How to connect the NUCLEO board to a PC.

Next, press the B1 USER button and the buttons connected to D2, D3, D4, D5, D6, and D7, one after the other. If the behavior of the LEDs is as described in Table 1.3 then the buttons that were connected are working fine and the program has been correctly loaded onto the NUCLEO board.



TIP: If the B1 USER button is working as described in Table 1.3 but one of the switches D2 to D7 is not working as expected, check the corresponding connections. If the B1 USER button is not working as described in Table 1.3 then the program has not been properly loaded onto the NUCLEO board.

As stated in the preface, in this book selected problems, named Examples, are tackled in seven steps. The first set of examples are presented below.

Example 1.1: Activate the Alarm When Gas is Detected

Objective

Write the first program of the book, load it onto the NUCLEO board, and introduce the *if* statement.

Summary of the Expected Behavior

LD1 must turn on when the button connected to pin D2 of the NUCLEO board is pressed and turn off when this button is released.

Test the Proposed Solution on the Board

Import the project “Example 1.1” using the URL available in [10], build the project, and drag the *.bin* file onto the NUCLEO board. Then press and release the button connected to pin D2 and look at the behavior of LD1. The LED should turn on when the button is pressed and turn off when the button is released.

Discussion of the Proposed Solution

In embedded systems there is a set of *statements* that are executed forever until the power supply is removed. Figure 1.9 illustrates the general structure, which is composed of two main parts:

- *Libraries, Definitions and Global Declarations and Initializations*, indicated in blue.
- *Implementation of the `main()` function*, indicated in green.

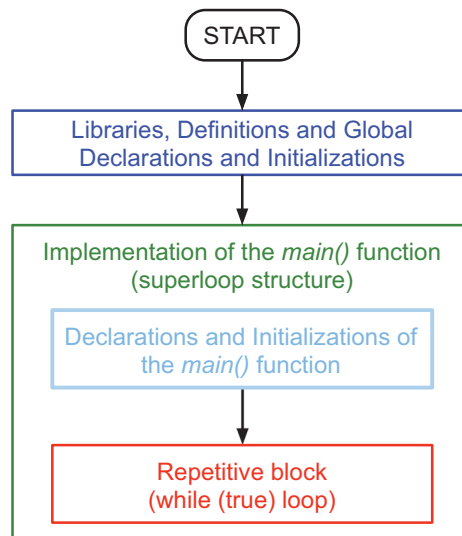


Figure 1.9 Structure of the proposed solution.

Libraries, Definitions and Global Declarations and Initializations includes the *software libraries* that are used. In addition, here some definitions of values can be established, and *global variables* can be declared and initialized, among other elements.

Implementation of the *main()* function consists of two parts, which are illustrated in Figure 1.9:

- *Declarations and Initializations of the `main()` function*, indicated in light blue.
- *Repetitive block*, indicated in red.

In Initializations of the *main()* function, the *inputs* and *outputs* of the board that are used are declared, and the initial states and other configurations are established.

The *repetitive block* is made by the statements that are executed repeatedly in the *while (true)* loop. This kind of structure is known as a *superloop*. That is, the code block corresponding to a *while* statement is executed as long as the condition of the *while* loop is true. If this condition is explicitly defined as *true* (i.e., *while (true)*) the code block of the *while* loop is executed forever.

Implementation of the Proposed Solution

The *main.cpp* file is where the main program is written in C or C++ language (the differences and history of C and C++ are discussed in Chapter 10 and in [11]). The *main.cpp* file of Example 1.1 is shown in Code 1.1. The parts corresponding to Figure 1.9 are highlighted in color. An explanation of each part is included over the code. In lines 1 and 2 of Code 1.1, `#include "mbed.h"` and `#include "arm_book_lib.h"` are used to instruct the Mbed Online Compiler to insert pre-written files that describe, among other things, the interfaces to external libraries that will then be included automatically by the tools, where *classes* and *macros* such as *DigitalIn*, *DigitalOut*, *ON*, and *OFF* are defined. The *main()* function is implemented from line 4 to line 23.

<pre> 1 #include "mbed.h" 2 #include "arm_book_lib.h" 3 4 int main() 5 { 6 DigitalIn gasDetector(D2); 7 8 DigitalOut alarmLed(LED1); 9 10 gasDetector.mode(PullDown); 11 12 alarmLed = OFF; 13 14 while (true) { 15 if (gasDetector == ON) { 16 alarmLed = ON; 17 } 18 19 if (gasDetector == OFF) { 20 alarmLed = OFF; 21 } 22 } 23 }</pre>	Include the libraries "mbed.h" and "arm_book_lib.h" - A digital input object (DigitalIn) named <i>gasDetector</i> is declared and assigned to D2. - A digital output object (DigitalOut) named <i>alarmLed</i> is declared and assigned to LED1. - <i>gasDetector</i> is configured with an internal pull-down resistor. - <i>alarmLed</i> (LED1) is assigned OFF. Do forever loop: - If <i>gasDetector</i> is active, <i>alarmLed</i> is assigned ON. - If <i>gasDetector</i> is not active, <i>alarmLed</i> is assigned OFF.
---	--

Code 1.1 Implementation of Example 1.1.



NOTE: Double straight quotes (") are used in lines 1 and 2 of Code 1.1. In the program codes that are introduced in this book, double and single straight quotes (') will gradually be introduced and used for different purposes. It is important for the reader to understand that double and single curly quotes ("", ' ', ') are different characters that cannot be used for the same purpose in the program codes. Similarly, a double straight quote cannot be replaced in the code by two consecutive straight single quotes.



NOTE: The colors used by different code editors to highlight reserved words and parts of the code may vary (i.e., some use green for *if* and others use blue). However, it has no relevance for the program code itself.

NOTE: C and C++ are free-form programming languages in which the positioning of characters in the program text is insignificant. However, to clarify the code structure in this book the Kernighan and Ritchie (K&R) indentation style [12] is used. The indentation rules of the K&R style are not discussed for the sake of brevity.

The instructions in line 6 and line 8 are to instruct the microcontroller that the button connected to D2 and LD1 (named LED1 in the code) will be used. The button will be used as an input, while LD1 will be used as an output (as indicated in Table 1.2). Line 6 declares a digital input with the name *gasDetector* and assigns it to button D2. Line 8 declares a digital output with the name *alarmLed* and assigns it to LED1. All these configurations are achieved by the *DigitalIn gasDetector(D2)* and *DigitalOut alarmLed(LED1)* declarations, where the corresponding *objects* are created.



NOTE: In this book, objects' names are stylized using *camel case* convention without capitalization of the first letter (known as *lower camel case* or *dromedary case*), as in *gasDetector* and *alarmLed*. In this way, these names can be easily differentiated from names defined by Mbed OS, like *DigitalIn*, *DigitalOut* or *PullDown*, which use the camel case convention with capitalization of the first letter.

In Line 10, *gasDetector.mode(PullDown)* is used to enable an internal pull-down resistor connected to the *DigitalIn* object *gasDetector*. A typical connection between a button and a microcontroller digital input based on a pull-down resistor is as shown on the left side of Figure 1.10. When the button is pressed, the NUCLEO board reads +3.3 V (high state). When the button is not pressed, the NUCLEO board reads 0 V (low state) because of the pull-down resistor ("R_pulldown"). In order to avoid connecting one resistor for each button, the internal pull-down resistors that are provided by the STM32 microcontroller are used in this example.

The right side of Figure 1.10 shows the simplified diagram of a typical connection between a button and a microcontroller digital input based on a pull-up resistor. In this case, when the button is pressed, the NUCLEO board reads a low state (0 V), and it reads a high state (3.3 V) when the button is not pressed. In the latter part of this book, pull-up resistors will be used so that the reader becomes familiar with them.

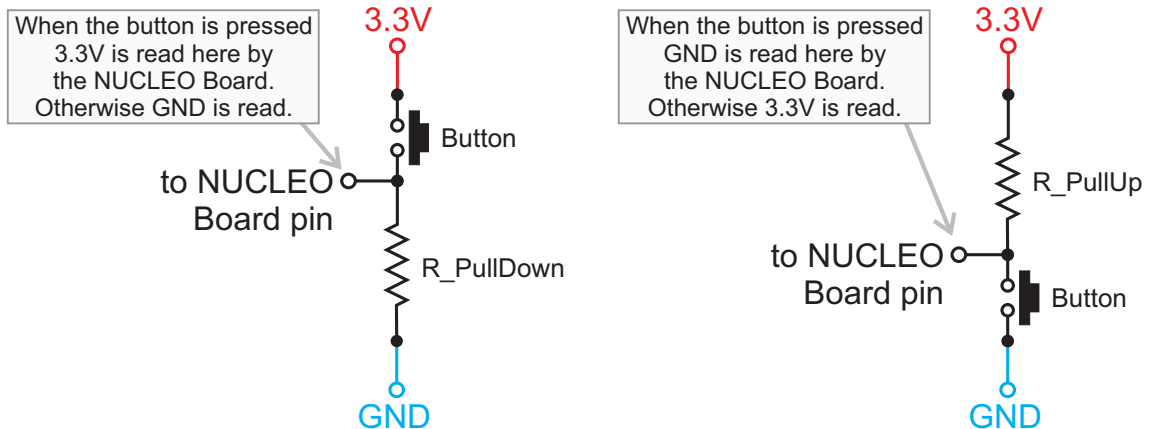


Figure 1.10 Conceptual diagram of how to connect a button to the NUCLEO board using pull-down and pull-up resistors.

In line 12, *alarmLed* is initialized to OFF. This instruction, together with lines 6 to 10, is highlighted in light blue in Code 1.1 and are called *Declarations and Initializations of the main() function*.

In line 14, the *while(true)* statement is used to indicate the beginning of the repetitive block that is repeated forever. In Example 1.3, the concept of a *Boolean variable* will be introduced, and this line will be explained in more detail.

Between lines 16 and 20, the following behavior is implemented: if the *gasDetector* is on (button connected to D2 is pressed) turn on the alarm (turn on LD1). If the *gasDetector* is off (button connected to D2 is not pressed) turn off the alarm (turn off LD1).



NOTE: To implement the comparison inside the *if* statements (line 15 and line 19) a double equals sign is used (i.e., “==”) while in order to assign values (line 16 and line 20) a single equals sign is used (i.e., “=”).

The reader should note that the braces “{” and “}” play a very important role in the code, because they are used to separate the code into parts.

The reader should also note that semicolons “;” are used to separate different statements, and parentheses “(” and “)” are used to indicate the beginning and ending of the arguments of the statements, as in *if()*, *while()*, and *main()*. Parentheses are also used to indicate which elements of the board are referred to with the objects *alarmLed* and *gasDetector*.

Proposed Exercises

1. What happens if one of the semicolons is replaced by a comma?
2. What happens if one of the *gasDetector* occurrences is replaced by *gasdetector*?

3. What can be changed in the code in order to use LD2 instead of LD1?
4. Is it possible to turn on LD1 and LD2 as the button connected to D2 is pressed?
5. What would happen if the *while (true)* statement was removed?
6. What will happen if the code in Table 1.4 is used?

Table 1.4 New proposed implementation for Example 1.1.

Lines in Code 1.1	New code to be used
<pre> 15 if (gasDetector == ON) { 16 alarmLed = ON; 17 } 18 19 if (gasDetector == OFF) { 20 alarmLed = OFF; 21 }</pre>	<pre> 16 if (gasDetector == ON) { 17 alarmLed = ON; 18 } else { 19 alarmLed = OFF; 20 } 21</pre>

Answers to the Exercises

1. The Online Compiler will not be able to compile the code and will indicate an error in the line where the semicolon was replaced by a comma.
2. The C/C++ code is case sensitive, so *gasdetector* will be interpreted as a different element than *gasDetector* and, as a consequence, an error will be indicated.
3. Line 8 should be changed to *DigitalOut alarmLed (LD2);*
4. Yes. The lines indicated in Table 1.5 should be changed.
5. The program will work once and will be over in a few microseconds.
6. The program will work in the same way as Code 1.1. The new code has the advantage of being more compact.

Table 1.5 Proposed modifications of the code in order to achieve the new behavior.

Lines in Code 1.1	New code to be used
8 <code>DigitalOut alarmLed(LED1);</code>	8 <code>DigitalOut alarmLed1(LED1);</code> 9 <code>DigitalOut alarmLed2(LED2);</code>
12 <code>alarmLed = OFF;</code>	13 <code>alarmLed1 = OFF;</code> 14 <code>alarmLed2 = OFF;</code>
17 <code>alarmLed = ON;</code>	19 <code>alarmLed1 = ON;</code> 20 <code>alarmLed2 = ON;</code>
20 <code>alarmLed = OFF;</code>	23 <code>alarmLed1 = OFF;</code> 24 <code>alarmLed2 = OFF;</code>

Example 1.2: Activate the Alarm on Gas Presence or Over Temperature

Objective

Review the *if* statement and introduce the OR operator.

Summary of the Expected Behavior

An LED must turn on/off as one or more buttons connected to the NUCLEO board are pressed or released.

Test the Proposed Solution on the Board

Import the project “Example 1.2” using the URL available in [10], build the project, and drag the *.bin* file onto the NUCLEO board. Then press and release the buttons connected to D2 or to D3 and look at the behavior of LD1. LD1 should turn on when any of the buttons connected to D2 and D3 are pressed. LD1 should turn off when both buttons connected to D2 and D3 are released.

Discussion of the Proposed Solution

In Example 1.1, the alarm (LD1) was activated when gas was detected (button connected to D2 was pressed). In this example, the over temperature detector is incorporated into the system and is simulated by the button connected to D3.

In Example 1.1, the *if* statement was used to implement the proposed behavior depending on the state of the button connected to D2. In this case, if the gas detector (D2) or the over temperature detector (D3) is active, the alarm should turn on. Otherwise, the alarm should turn off. The logical OR behavior is indicated by means of the `||` operator in C/C++.

Figure 1.11 shows the proposed parts to implement the solution, following the structure proposed in Figure 1.9. In the *Libraries, Definitions and Global Declarations and Initializations* (Figure 1.11 [a]), the *mbed.h* and *arm_book_lib.h* libraries are included. In the *Declarations and Initializations of the main() function* (Figure 1.11 [b]), the digital inputs *gasDetector* and *overTempDetector* are declared and assigned to D2 and D3, and a digital output named *alarmLed* is declared and assigned to LD1. The repetitive block (Figure 1.11 [c]) turns on *alarmLed* if *gasDetector* or *overTempDetector* is pressed and turns off *alarmLed* if both are not pressed.

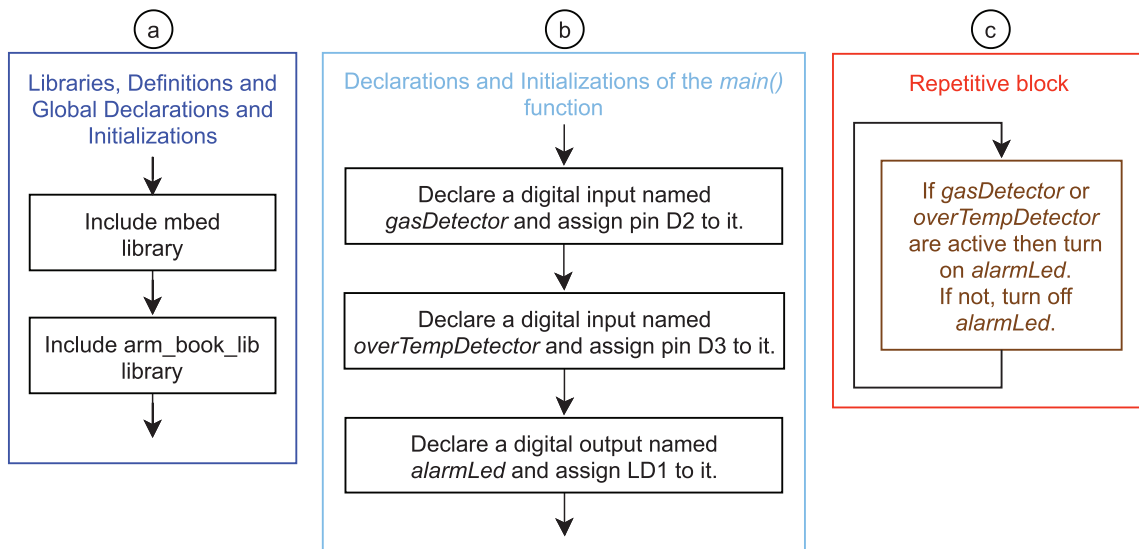


Figure 1.11 Main parts of the program of Example 1.2.

Figure 1.12 shows the details of the repetitive block. This block turns `alarmLed` (LD1) on if `gasDetector` or `overTempDetector` is active (buttons connected to D2 or D3 are pressed).

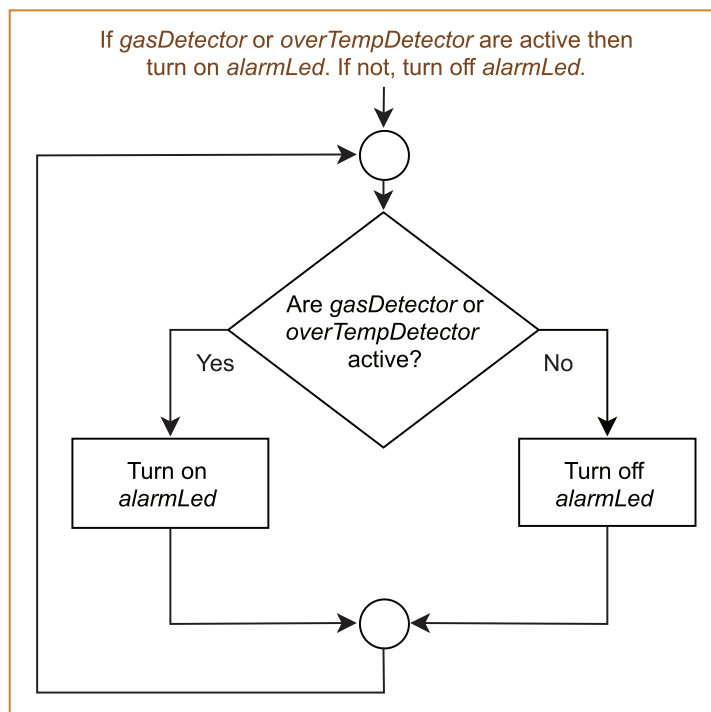


Figure 1.12 Details of the blocks that make up the repetitive block of Example 1.2.

Implementation of the Proposed Solution

In Code 1.2, the implementation of the proposed solution is presented. The parts corresponding to Figure 1.11 and Figure 1.12 are highlighted in color. An explanation of each part is included beside the code.

```

1 #include "mbed.h"
2 #include "arm_book_lib.h"
3
4 int main()
5 {
6     DigitalIn gasDetector(D2);
7     DigitalIn overTempDetector(D3);
8
9     DigitalOut alarmLed(LED1);
10
11     gasDetector.mode(PullDown);
12     overTempDetector.mode(PullDown);
13
14     while ( true ) {
15
16         if ( gasDetector || overTempDetector ) {
17             alarmLed = ON;
18         }
19         alarmLed = OFF;
20     }
21 }
22 }

```

Include the libraries "mbed.h" and "arm_book_lib.h"

- Digital input objects named *gasDetector* and *overTempDetector* are declared and assigned to D2 and D3, respectively.
- Digital output object named *alarmLed* is declared and assigned to LED1.
- *gasDetector* and *overTempDetector* are configured with internal pull-down resistors.

Do forever loop:

- If *gasDetector* or *overTempDetector* are active, *alarmLed* is assigned ON.
- Else, *alarmLed* is assigned OFF.

Code 1.2 Implementation of Example 1.2.

Proposed Exercises

1. What should be modified in order to simulate the alarm by means of LD2?
2. How can it be implemented so that when gas presence is detected or over temperature is detected LD1 and LD2 are both turned on?
3. How can line 16 of Example 1.2 be modified in order to use an explicit formulation of the condition, as in Example 1.1?
4. Is there another way to indicate the OR logical operator beside the || symbols used in Example 1.6?

Answers to the Exercises

1. In line 8, LED1 should be replaced by LED2.
2. It can be achieved by means of the changes in Table 1.6.

Table 1.6 Proposed modifications in the code in order to achieve the new behavior.

Lines in Code 1.2	New code to be used
9 DigitalOut alarmLed(LED1);	9 DigitalOut alarmLed1(LED1); 10 DigitalOut alarmLed2(LED2);
17 alarmLed = ON;	18 alarmLed1 = ON; 19 alarmLed2 = ON;
19 alarmLed = OFF;	21 alarmLed1 = OFF; 22 alarmLed2 = OFF;

3. Line 16 can be rewritten as follows:

```
if ( gasDetector == ON || overTempDetector == ON )
```

4. The logical operator OR can be expressed as follows

```
if ( gasDetector == ON or overTempDetector == ON )
```



NOTE: In this book, the notation `||` is used because of historical reasons. However, in Mbed OS, `or` can be used to indicate the logical operator OR; in addition, other reserved words such as *not*, *and*, etc. can be used.

Example 1.3: Keep the Alarm Active After Gas or Over Temperature Were Detected

Objective

Learn how to use a Boolean variable to keep track of the state of a given element.

Summary of the Expected Behavior

The LED is turned on when one or more of the alarm conditions are activated (simulated by buttons connected to the NUCLEO board) and remains on until those alarm conditions are removed and another button is pressed.

Test the Proposed Solution on the Board

Import the project “Example 1.3” using the URL available in [10], build the project, and drag the `.bin` file onto the NUCLEO board. Then press and release the buttons connected to D2 and D3 and look at the behavior of LD1. LD1 should turn on when the buttons connected to D2 and D3 are pressed and turn off when the B1 USER button is pressed.

Discussion of the Proposed Solution

In Example 1.2, the alarm was directly controlled by the buttons connected to D2 and D3 that simulated the gas detector and the over temperature detector, respectively. When any of these buttons were pressed the alarm was turned on, and when both buttons were released the alarm was turned off. To keep the alarm activated until another condition occurs, the alarm state must be stored. This can be done by means of a *Boolean variable*. In this scheme, the activation of the alarm is determined by the state of the variable *alarmState*. This variable is assigned ON when the gas detector or the over temperature detector is activated and is assigned OFF when the B1 USER button is pressed.



NOTE: Strictly speaking, a Boolean variable has only two valid states: *true* or *false*. However, for practical purposes, we will use ON and OFF to refer to the values *true* and *false* of the variable *alarmState*.

It is worth noting that the *while(true)* instruction used to indicate the beginning of the repetitive block is based on the idea that what is inside the braces of the *while* statement should be repeated until the condition inside the parentheses of the *while* becomes false. Given that it is written *while(true)*, it will never be false. Therefore, the instructions inside the braces will be repeated forever.

Figure 1.13 shows the proposed main parts to implement the solution of this example. In the *Libraries, Definitions and Global Declarations and Initializations* (Figure 1.13 [a]), the *mbed.h* and *arm_book_lib.h* libraries are included. In the *Declarations and Initializations of the main() function* (Figure 1.13 [b]), the digital inputs *gasDetector*, *overTempDetector*, and *alarmOffButton* are declared and assigned to buttons D2, D3, and B1 USER, respectively, the digital output *alarmLed* is declared and assigned to LD1, and the variable *alarmState* is declared and initialized to OFF.

The *repetitive block* (Figure 1.13 [c]) is made up of three blocks: a block indicated in brown, which assigns ON to *alarmState* if *gasDetector* or *overTempDetector* is activated; a block indicated in orange, which assigns the value of *alarmState* to *alarmLed*; and a block indicated in violet, which assigns OFF to *alarmState* if the *alarmOffButton* is pressed.



NOTE: In this book names of variables are stylized using the lower camel case format, as in *alarmState*.

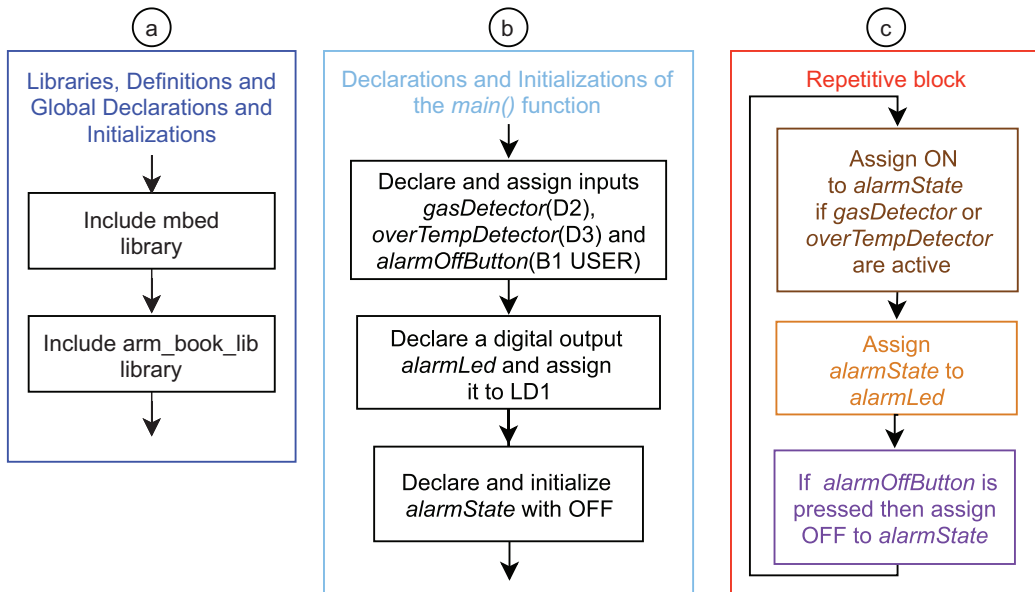


Figure 1.13 Main parts of the program of Example 1.3.

Figure 1.14 shows the details of the blocks that compose the repetitive block. Figure 1.14 (a) shows the block that assigns ON to *alarmState* if *gasDetector* or *overTempDetector* is active. Figure 1.14 (b) shows the details of the block that assigns *alarmState* to *alarmLed*. Figure 1.14 (c) shows the details of the block that assigns OFF to *alarmState* if *alarmOffButton* is pressed.

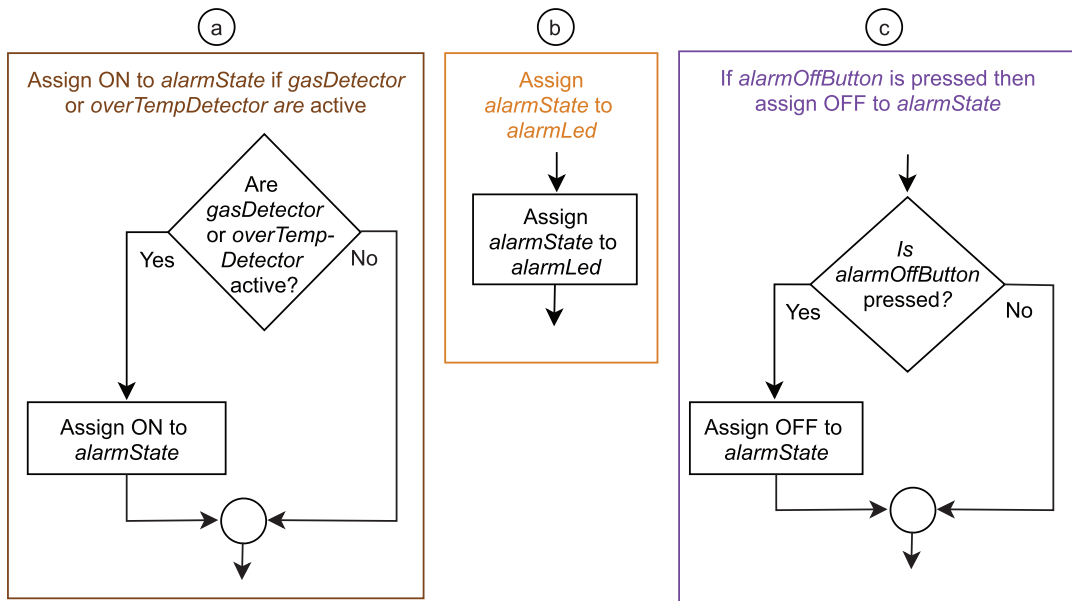


Figure 1.14 Details of the blocks that compose the repetitive block of Example 1.3.

Implementation of the Proposed Solution

In Code 1.3 the implementation of the proposed solution is presented. The parts corresponding to Figure 1.13 and Figure 1.14 are highlighted in color. An explanation of each part is included beside the code.



NOTE: In the code, “BUTTON1” is used to refer to the “B1 USER button” because it is defined this way in the mbed.h library. For the same reason, “LED1” is used to refer to “LD1”, such as in Examples 1.1 and 1.2.

```

1 #include "mbed.h"
2 #include "arm_book_lib.h"
3
4 int main()
5 {
6     DigitalIn gasDetector(D2);
7     DigitalIn overTempDetector(D3);
8     DigitalIn alarmOffButton(BUTTON1);
9
10    DigitalOut alarmLed(LED1);
11
12    gasDetector.mode(PullDown);
13    overTempDetector.mode(PullDown);
14
15    alarmLed = OFF;
16
17    bool alarmState = OFF;
18
19    while ( true ) {
20
21        if ( gasDetector || overTempDetector ) {
22            alarmState = ON;
23        }
24
25        alarmLed = alarmState;
26
27        if ( alarmOffButton ) {
28            alarmState = OFF;
29        }
30    }
31 }

```

Include the libraries "mbed.h" and "arm_book_lib.h"

- Digital input objects named *gasDetector*, *overTempDetector* and *alarmOffButton*, are declared and assigned to D2, D3, and BUTTON1 (B1 USER), respectively.
- Digital output object named *alarmLed* is declared and assigned to LED1.
- *gasDetector* and *overTempDetector* are configured with internal pull-down resistors.
- *alarmLed* is assigned OFF.
- *alarmState* is declared and assigned OFF.

Do forever loop:

- If *gasDetector* or *overTempDetector* are active, assign *alarmState* with ON.
- Assign *alarmLed* with *alarmState*.
- If *alarmOffButton*, assign *alarmState* with OFF.

Code 1.3 Implementation of Example 1.3.



NOTE: It is not necessary to enable an internal pull-down resistor for the B1 User button. This is because this button is already connected to an external pull-down resistor placed in the NUCLEO board, as well as to other elements that are used to reduce the *electrical noise* of the signal that is supplied to the microcontroller.

Proposed Exercises

1. How can the code be changed in such a way that LD1 is turned on at the beginning, is turned off when gas presence or over temperature is detected, and is turned on when the B1 USER button is pressed?
2. What should be modified in order to turn off the alarm by means of the button connected to D4?

Answers to the Exercises

1. It can be achieved by means of the changes shown in Table 1.7.
2. In line 16, BUTTON1 should be replaced by D4.

Table 1.7 Proposed modifications in the code in order to achieve the new behavior.

Lines in Code 1.3	New code to be used
17 bool alarmState = OFF	17 bool alarmState = ON
22 alarmState = ON;	22 alarmState = OFF;
28 alarmState = OFF;	28 alarmState = ON;

Example 1.4: Secure the Alarm Deactivation by Means of a Code

Objective

Introduce the AND and NOT operators.

Summary of the Expected Behavior

An LED must turn on when one or more buttons connected to the NUCLEO board are pressed, and it must be kept on until a combination of certain buttons is entered.

Test the Proposed Solution on the Board

Import the project “Example 1.4” using the URL available in [10], build the project, and drag the *.bin* file onto the NUCLEO board. Then press and release either of the buttons connected to D2 or D3. Look at the behavior of LD1. LD1 should turn on when any of the buttons connected to D2 and D3 are pressed and turn off only when the buttons connected to D4 and D5 are pressed together at the same time.

Discussion of the Proposed Solution

In general, the behavior of this code is similar to the code in Example 1.3, but in this example the way the alarm is turned off is different. In this case, a combination of certain buttons must be pressed simultaneously. To turn off the alarm, the buttons connected to D4 *and* D5 should be pressed while at

the same time the buttons connected to D6 and D7 should *not* be pressed. In C/C++, the logical AND is indicated by means of the && operator and the logical operator NOT is indicated by means of the ! operator.

Figure 1.15 shows the proposed main parts to implement the solution of this example. In the *Libraries, Definitions and Global Declarations and Initializations* (Figure 1.15 [a]), the *mbed.h* and *arm_book_lib.h* libraries are included. In the *Declarations and Initializations of the main() function* (Figure 1.15 [b]), the pins connected to D2, D3, D4, D5, D6 and D7 are configured as inputs, the pin connected to LD1 is configured as output, and the variable *alarmState* is declared and initialized to OFF.

The repetitive block (Figure 1.15 [c]) is made up of three blocks: a block indicated in brown, which assigns ON to *alarmState* if *gasDetector* or *overTempDetector* is activated; a block indicated in orange, which assigns the value of *alarmState* to *alarmLed*; and a block indicated in violet, which assigns OFF to *alarmState* if the buttons connected to D4 and D5 are pressed while the buttons connected to D6 and D7 are not pressed.

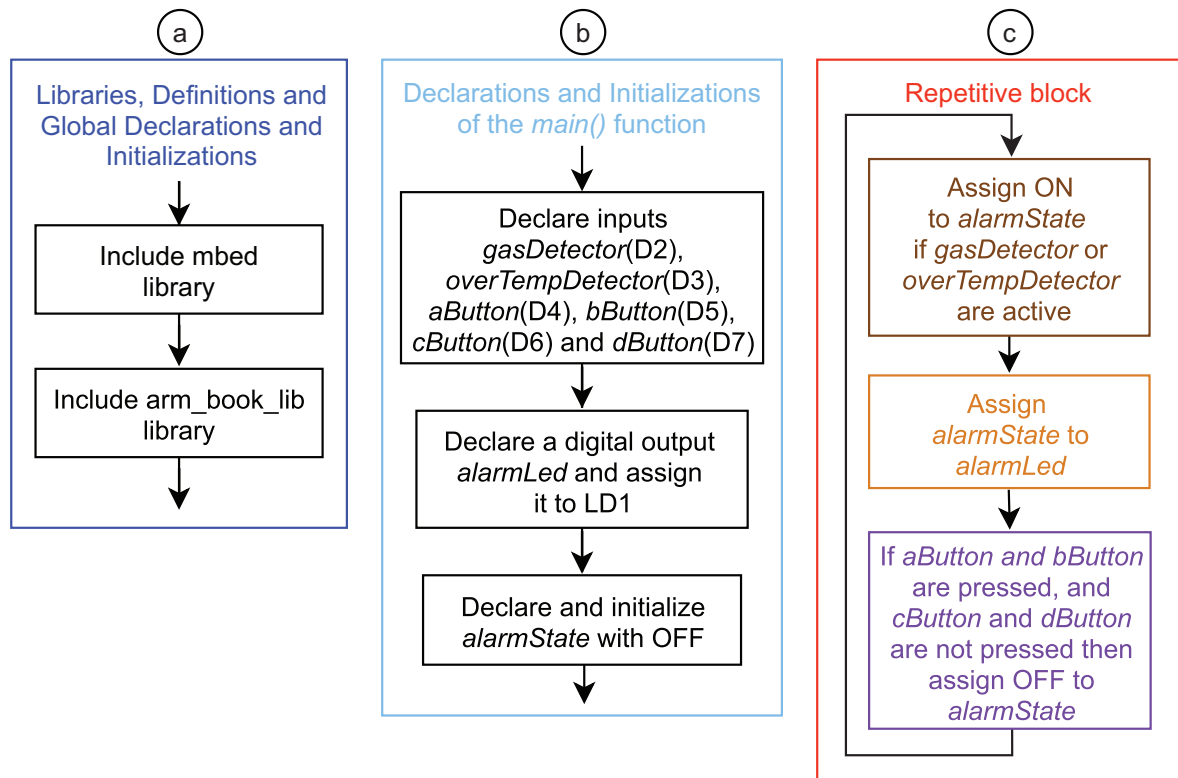


Figure 1.15 Main parts of the program of Example 1.4.

Figure 1.16 shows the details of the blocks that compose the repetitive block. In this case, only the block indicated in violet is included because the brown and orange blocks are the same as in Figure 1.16 (a) and Figure 1.16 (b), respectively. Figure 1.16 (c) shows the block that assigns OFF to *alarmState* if the buttons connected to D4 and D5 are pressed and the buttons connected to D6 and D7 are not pressed.

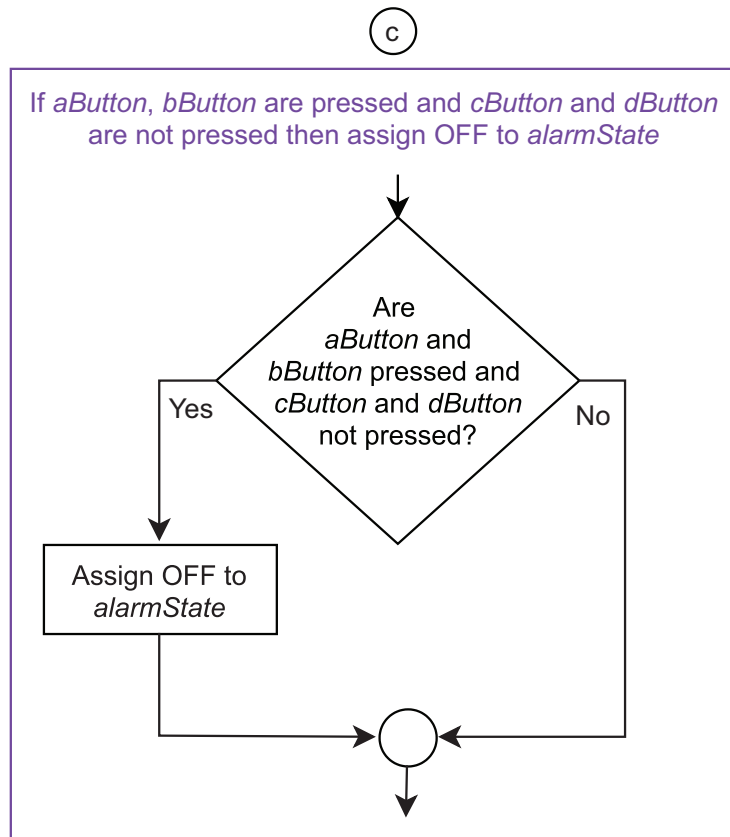


Figure 1.16 Details of the blocks that make up the repetitive block of Example 1.4.

Implementation of the Proposed Solution

In Code 1.4 the implementation of the proposed solution is presented. The parts corresponding to Figure 1.15 and Figure 1.16 are highlighted in different colors. An explanation of each part is also included beside the code.

```

1 #include "mbed.h"
2 #include "arm_book_lib.h"
3

```

Include the libraries "mbed.h" and "arm_book_lib.h"

```

4 int main()
5 {
6     DigitalIn gasDetector(D2);
7     DigitalIn overTempDetector(D3);
8     DigitalIn aButton(D4);
9     DigitalIn bButton(D5);
10    DigitalIn cButton(D6);
11    DigitalIn dButton(D7);
12
13    DigitalOut alarmLed(LED1);
14
15    gasDetector.mode(PullDown);
16    overTempDetector.mode(PullDown);
17    aButton.mode(PullDown);
18    bButton.mode(PullDown);
19    cButton.mode(PullDown);
20    dButton.mode(PullDown);
21
22    alarmLed = OFF;
23
24    bool alarmState = OFF;
25
26    while (true) {
27        if ( gasDetector || overTempDetector ) {
28            alarmState = ON;
29        }
30
31        alarmLed = alarmState;
32
33        if ( aButton && bButton && !cButton && !dButton ) {
34            alarmState = OFF;
35        }
36    }
37 }
38 }

```

- Digital input objects named *gasDetector*, *overTempDetector* and *aButton* to *dButton*, are declared and assigned to D2 to D7, respectively.
- Digital output object named *alarmLed* is declared and assigned to LED1.
- *gasDetector*, *overTempDetector* and *aButton* to *dButton* are configured with internal pull-down resistors.
- *alarmLed* is assigned OFF.
- *alarmState* is declared and assigned OFF.

Do forever loop:

- If *gasDetector* or *overTempDetector* is pressed, assign *alarmState* with ON.
- Assign *alarmLed* with *alarmState*.
- If *aButton* and *bButton* are pressed, and neither *cButton* nor *dButton* are pressed, assign *alarmState* with OFF.

Code 1.4 Implementation of Example 1.4.

Proposed Exercises

1. How can the code be changed in such a way that the alarm is turned off by means of pressing the buttons connected to D6 and D7, while the buttons connected to D4 and D5 are not pressed?
2. How can the code be changed in such a way that the alarm is turned off by means of pressing the buttons connected to D4, D5, and D6, while the button connected to D7 is not pressed?

Answers to the Exercises

1. It can be achieved by means of the changes in Table 1.8.
2. It can be achieved by means of the changes in Table 1.9.

Table 1.8 Proposed modifications in the code in order to achieve the new behavior.

Lines in Code 1.4	New code to be used
34 if (aButton && bButton && !cButton && !dButton)	34 if (!aButton && !bButton && cButton && dButton)

Table 1.9 Proposed modifications in the code in order to achieve the new behavior.

Lines in Code 1.4	New code to be used
34 if (aButton && bButton && !cButton && !dButton)	34 if (aButton && bButton && cButton && !dButton)

Example 1.5: Block the System when Five Incorrect Codes are Entered

Objective

Introduce nested *ifs* and the usage of non-Boolean variables to count the number of iterations.

Summary of the Expected Behavior

If five wrong passwords are introduced, the system is blocked.

Test the Proposed Solution on the Board

Import the project “Example 1.5” using the URL available in [10], build the project, and drag the *.bin* file onto the NUCLEO board. Then press and release the buttons connected to D2 or to D3 and look at the behavior of LD1. LD1 should turn on. If buttons A, B, and Enter (D4, D5, and B1 USER Button) are pressed, LD1 will turn off. Again, press and release the buttons connected to D2 or to D3. LD1 should turn on. If another combination of buttons is pressed (for instance A and B1 USER) the LD3 (Incorrect code) will turn on. A new combination of buttons can be tried after all four buttons connected to D4–D7 are pressed simultaneously. The fifth time that an incorrect combination of buttons is entered, LD2 (System blocked) will turn on, indicating that the system has been blocked. Press the reset button to reset the NUCLEO board and turn off the System blocked LED.

A summary of the buttons that have to be pressed in each case is shown in Table 1.10.

Table 1.10 Summary of the smart home system buttons that should be pressed in each case.

Functionality	Buttons that should be pressed	Corresponding DigitalIn
To test if a given code turns off the Alarm LED	Any of A, B, C, and/or D + Enter	(D4, D5, D6, D7) + BUTTON1
The correct code that turns off the Alarm LED	A + B + Enter	D4 + D5 + BUTTON1
To turn off the Incorrect code LED and enable a new attempt to turn off the Alarm LED	A + B + C + D	D4 + D5 + D6 + D7
To turn off the System Blocked LED (that is turned on after entering five incorrect passwords)	No buttons available (power should be removed or B2 RESET button pressed)	

Discussion of the Proposed Solution

In Example 1.4 there was no limit to the number of incorrect combinations of buttons that could be entered. In this example, the system is blocked when five incorrect combinations are entered. In order to count the number of incorrect combinations entered, *non-Boolean* variables are introduced. Non-Boolean variables can be used to store integer or non-integer numbers depending on the specific type used, as discussed below.

Figure 1.17 shows the proposed main parts to implement the solution of this example. In the *Libraries, Definitions and Global Declarations and Initializations* (Figure 1.17 [a]), the *mbed.h* and *arm_book_lib.h* libraries are included. In the *Declarations and Initializations of the main() function* (Figure 1.17 [b]), several inputs are declared and assigned to D2, D3, D4, D5, D6, D7, and BUTTON1; several outputs are declared and assigned to LD1, LD2, and LD3; the variable *alarmState* is declared and initialized to OFF; and the variable *numberOfIncorrectCodes* is declared and initialized to 0.

The repetitive block (Figure 1.17 [c]) is made up of three blocks: a block indicated in brown, which assigns ON to *alarmState* if *gasDetector* or *overTempDetector* is active; a block indicated in orange, which assigns the value of *alarmState* to *alarmLed*; and a block indicated in violet. In the violet block, if *numberOfIncorrectCodes* has reached five, the system is blocked. If *numberOfIncorrectCodes* has not reached five, then it checks if the user wants to enter a new code to turn off the alarm. If so, it checks if the entered code is correct in order to turn off the alarm LED. The *incorrectCodeLed* (LD2) is turned on if an incorrect code is entered.

Figure 1.18 shows the details of the blocks that comprise the repetitive block. In this case, only the block indicated in violet is included because the brown and orange blocks are the same as in Figure 1.14 (a) and Figure 1.14 (b), respectively.

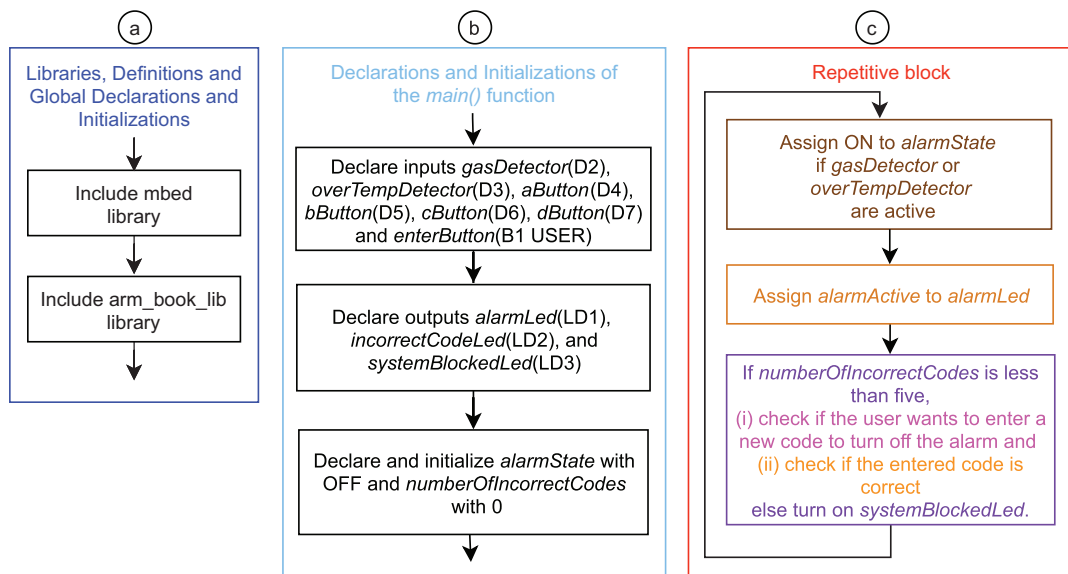


Figure 1.17 Main parts of the program of Example 1.5.

Figure 1.18 shows that the first step is to evaluate if *numberOfIncorrectCodes* is less than five. If five or more incorrect codes have been entered, then the system is blocked, and the System blocked LED is turned on. Buttons are not checked any more, and the only way to unblock the system is to turn off the power supply.

If *numberOfIncorrectCodes* is less than five, a check is made to see if buttons A, B, C, and D are all pressed at the same time, while the Enter button is not pressed, which means that the user wants to enter a code to turn off the alarm. In this case, the Incorrect code LED is turned off.

Next, it is assessed whether the following conditions are all accomplished at the same time: i) the Enter button is being pressed, ii) the Incorrect code LED is OFF, iii) the alarm is ON. In this way, it is determined if the user wants to enter a code (recall Table 1.10), and that a code can be entered (i.e., Incorrect code LED is off) and that a code must be entered (the alarm is ON).

If that is the case, to turn off the Alarm LED, buttons A and B should be pressed, while buttons C and D are not pressed. In this case, OFF is assigned to *alarmState* and *numberOfIncorrectCodes* is set to 0.

If any other combination of buttons A, B, C, and D is entered then the Incorrect code LED is turned on and *numberOfIncorrectCodes* is incremented by one.



NOTE: It is important to mention that this is the first time in this book that an *if* condition is evaluated inside of another *if* condition. This grouping of *if* statements is known as *nested ifs*.



WARNING: Two *if* conditions can also be evaluated one after the other, no matter the result of the first *if* condition. In this case, the structure does not correspond to a nested *if*.

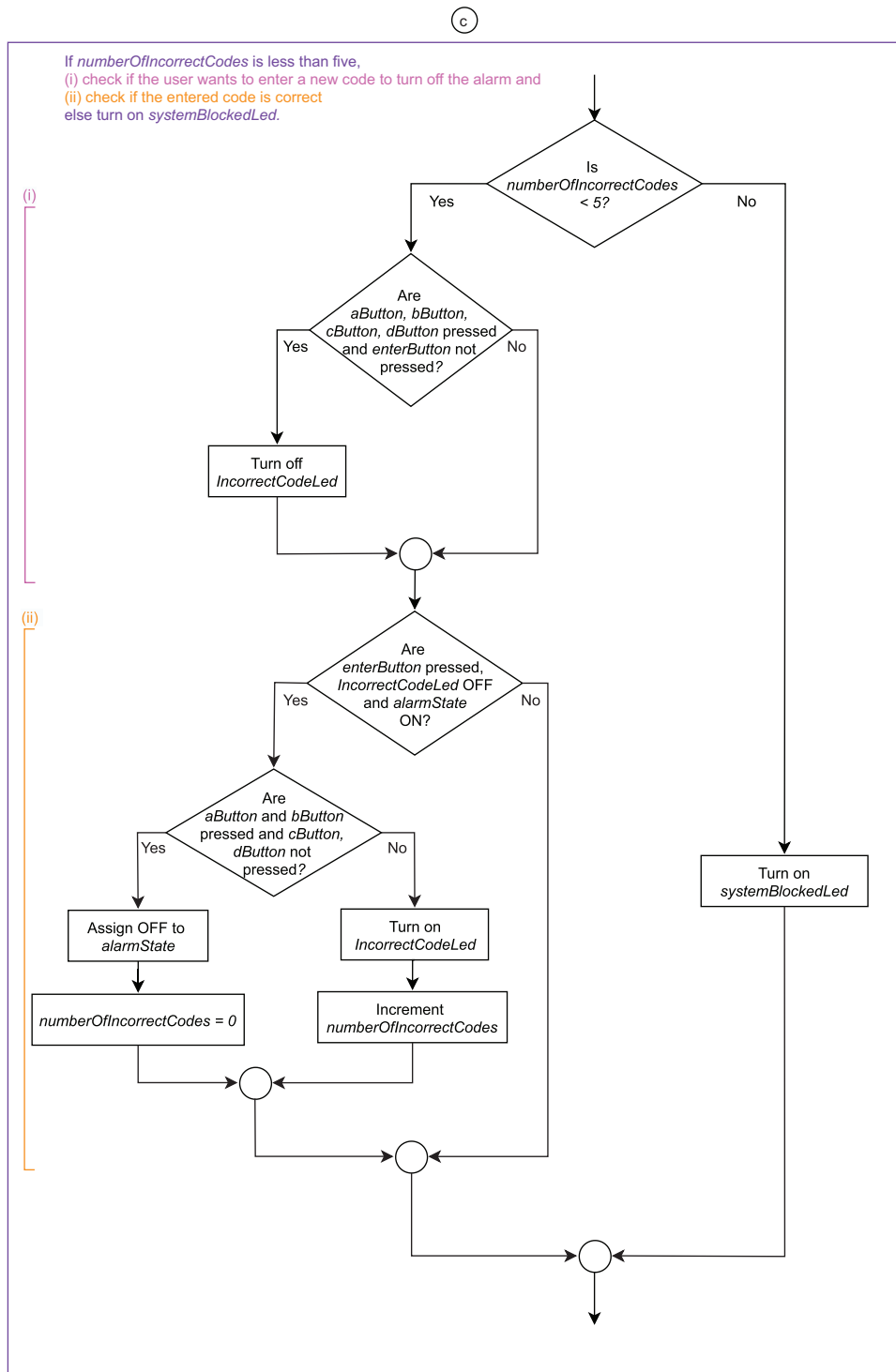


Figure 1.18 Details of the blocks that make up the repetitive block of Example 1.5.

Implementation of the Proposed Solution

In Code 1.5 the implementation of the proposed solution is presented. The parts corresponding to Figure 1.17 and Figure 1.18 are highlighted in different colors. An explanation of each part is also included beside the source code.

It is important to emphasize that all the considerations regarding the nested *if* discussed above correspond to lines 40 to 58 in Code 1.5. These lines are not discussed here because their behavior has already been discussed in detail in the explanation regarding Figure 1.18.

```

1 #include "mbed.h"
2 #include "arm_book_lib.h"
3
4 int main()
5 {
6     DigitalIn enterButton(BUTTON1);
7     DigitalIn gasDetector(D2);
8     DigitalIn overTempDetector(D3);
9     DigitalIn aButton(D4);
10    DigitalIn bButton(D5);
11    DigitalIn cButton(D6);
12    DigitalIn dButton(D7);
13
14    DigitalOut alarmLed(LED1);
15    DigitalOut incorrectCodeLed(LED3);
16    DigitalOut systemBlockedLed(LED2);
17
18    gasDetector.mode(PullDown);
19    overTempDetector.mode(PullDown);
20    aButton.mode(PullDown);
21    bButton.mode(PullDown);
22    cButton.mode(PullDown);
23    dButton.mode(PullDown);
24
25    alarmLed = OFF;
26    incorrectCodeLed = OFF;
27    systemBlockedLed = OFF;
28
29    bool alarmState = OFF;
30    int numberOfIncorrectCodes = 0;
31
32    while (true) {
33
34        if ( gasDetector || overTempDetector ) {
35            alarmState = ON;
36        }
37
38        alarmLed = alarmState;
39
40        if ( numberOfIncorrectCodes < 5 ) {
41
42            if ( aButton && bButton && cButton && dButton && !enterButton ) {
43                incorrectCodeLed = OFF;
44            }
45
46            if ( enterButton && !incorrectCodeLed && alarmState) {
47                if ( aButton && bButton && !cButton && !dButton ) {
48                    alarmState = OFF;
49                    numberOfIncorrectCodes = 0;
50                } else {
51                    incorrectCodeLed = ON;
52                    numberOfIncorrectCodes = numberOfIncorrectCodes + 1;
53                }
54            }
55            else {
56                systemBlockedLed = ON;
57            }
58        }
59    }
60 }

```

Include the libraries "mbed.h" and "arm_book_lib.h"

- Digital input objects named *gasDetector*, *overTempDetector* and *aButton* to *dButton*, are declared and assigned to D2 to D7, respectively.
- Digital output objects named *alarmLed*, *incorrectCodeLed*, and *systemBlockedLed* are declared and assigned to LED1, LED2 and LED3, respectively.
- *gasDetector*, *overTempDetector* and *aButton* to *dButton* are configured with internal pull-down resistors.
- *alarmLed*, *incorrectCodeLed*, and *systemBlockedLed* are assigned OFF.
- *alarmState* is declared and assigned OFF.
- *numberOfIncorrectCodes* is declared and assigned 0 (zero).

Do forever loop:

- If *gasDetector* or *overTempDetector* are active, assign *alarmState* with ON.
- Assign *alarmLed* with *alarmState*.
- If *numberOfIncorrectCodes* is less than five, implement the logic (discussed in the text) in order to determine the state of *incorrectCodeLed* and if *alarmState* must be assigned with OFF, and if *numberOfIncorrectCodes* must be set to 0 (zero) or increased by 1.
- If *numberOfIncorrectCodes* is greater than or equal to five, assign *systemBlockedLed* with ON.

Code 1.5 Implementation of Example 1.5.

Proposed Exercise

1. How can the code be changed in such a way that the system is blocked after three incorrect codes are entered?

Answer to the Exercise

1. It can be achieved by means of the change in Table 1.11.

Table 1.11 Proposed modification in the code in order to achieve the new behavior.

Line in Code 1.5	New code to be used
40 if (numberOfIncorrectCodes < 5)	40 if (numberOfIncorrectCodes < 3)

1.3 Under the Hood

1.3.1 Brief Introduction to the Cortex-M Processor Family and the NUCLEO Board

In this chapter, many programs were developed using the NUCLEO board, provided with the STM32F429ZIT6U microcontroller. This microcontroller is manufactured by STMicroelectronics [13] using a Cortex-M4 processor designed by Arm Ltd. [14]. In order to support some concepts that were introduced through this chapter, as well as concepts that will be presented in the following chapters, in this subsection a brief introduction to Cortex-M processors and some details on the NUCLEO board are provided.

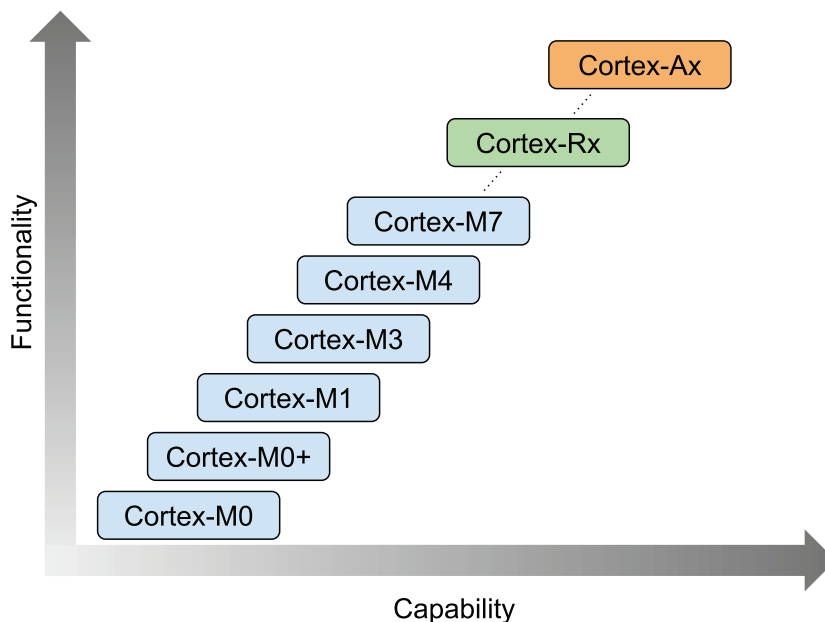


Figure 1.19 Simplified diagram of the Cortex processor family.

In Figure 1.19, a simplified diagram of the Cortex family of processors is shown. The Cortex-M processors are the most energy-efficient embedded devices of the family and have the lowest cost; therefore, they are typically used in microcontrollers and sensors. The Cortex-Rx are robust real-time performance processors, ideal for automotive and safety-critical applications. The Cortex-Ax processors have supreme performance and are used in smartphones, computers, and high-end microprocessors.



NOTE: The Cortex-M family currently has more than ten members (a list is available from [15]), and only six of them are shown in the diagram. In addition, the “Cortex-Rx” and “Cortex-Ax” frames represent whole families, which at the time of writing have half a dozen members and over twenty members, respectively.

In this Under the Hood section, only the Cortex-M0, Cortex-M3, and Cortex-M4 processors are analyzed, because they are more likely to be the processors used in the microcontrollers for elementary embedded systems projects. These processors are primarily focused on delivering highly deterministic behavior in a wide range of power-sensitive applications.

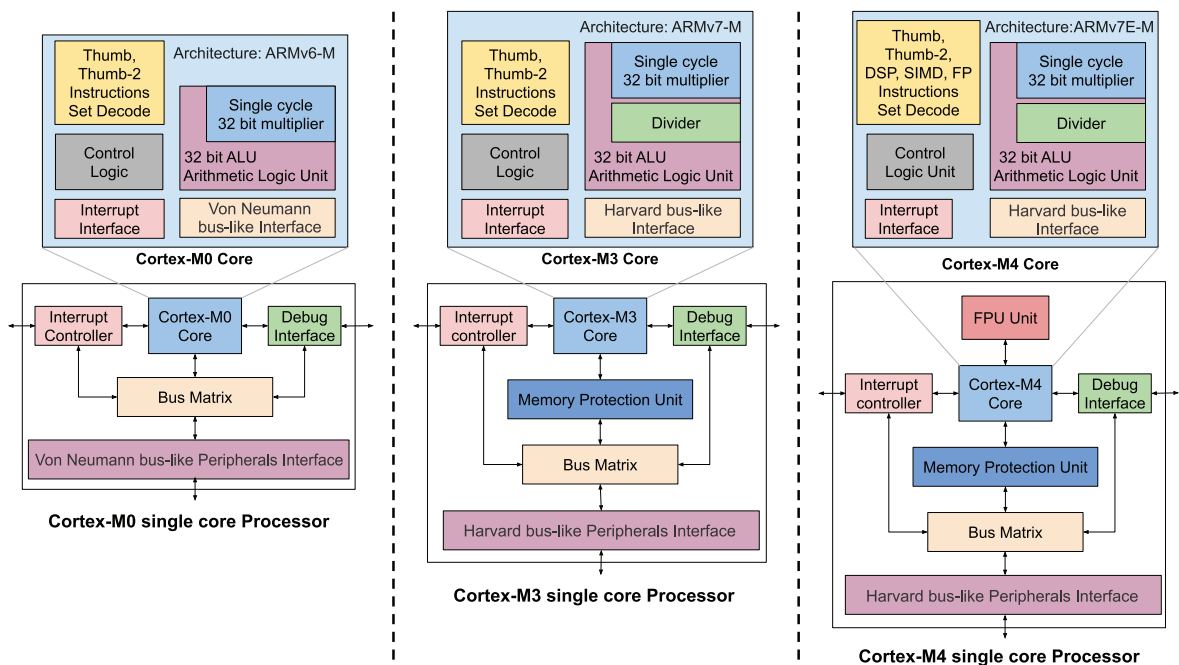


Figure 1.20 Simplified diagram of the Cortex M0, M3, and M4 processors, and details of the corresponding cores.

In Figure 1.20, diagrams of the Cortex-M0, M3, and M4 processors are shown, as well as some details of their corresponding cores. It can be appreciated how there are some similarities among the processors, such as interruption handling (a topic that will be discussed later in this book), and some differences, for example:

- The Cortex-M0 is Von Neumann bus-like (one bus for data/instruction), while the Cortex-M3 and M4 are Harvard bus-like (data and instruction in separate buses), which allows for faster communication.

- The Cortex-M3 and M4 processors have a Memory Protection Unit, which allows safer operation.
- The Cortex-M4 has a larger set of instructions and has a Floating Point Unit (FPU), which speeds up certain calculations because the FPU is used instead of using a multitude of elementary operations.

To indicate the complexity of the different cores, Figure 1.21 shows the set of instructions that each of the cores can process. This is called the *Instruction Set Architecture (ISA)*. It can be seen that the Cortex-M0 core implements a reduced number of instructions, while the Cortex-M3 core handles those instructions and incorporates many more instructions, and the Cortex-M4 incorporates even more. Instructions added in the Cortex-M7 are shown in Figure 1.21 to stress that more advanced and powerful cores are available in the Cortex-M family, as shown in [16]. Those cores are not used in this book, hence are not discussed here.



Figure 1.21 Arm Cortex M0, M3, and M4 Instruction Set Architecture (ISA).

The instructions shown in Figure 1.21 are in assembly language (also called assembler language), which is not as easy to read and write for programmers as the C/C++ language. Thus, a program called a *compiler* is used to automatically translate C/C++ language code into the corresponding assembly language code. The compiler also verifies that the C/C++ language code complies with the language rules defined in the standards. If there is a syntax issue, the compiler cannot generate the assembly code and an error is indicated to the programmer. Keil Studio Cloud includes all these features.

36

The STM32F429ZIT6U microcontroller includes a Cortex-M4 processor, as shown in Figure 1.22. It can be appreciated that, beyond the processor, the microcontroller includes other peripherals such as communication cores (ethernet, USB, UART, etc.), memory, timers, and GPIO (General Purpose Input Output) ports.

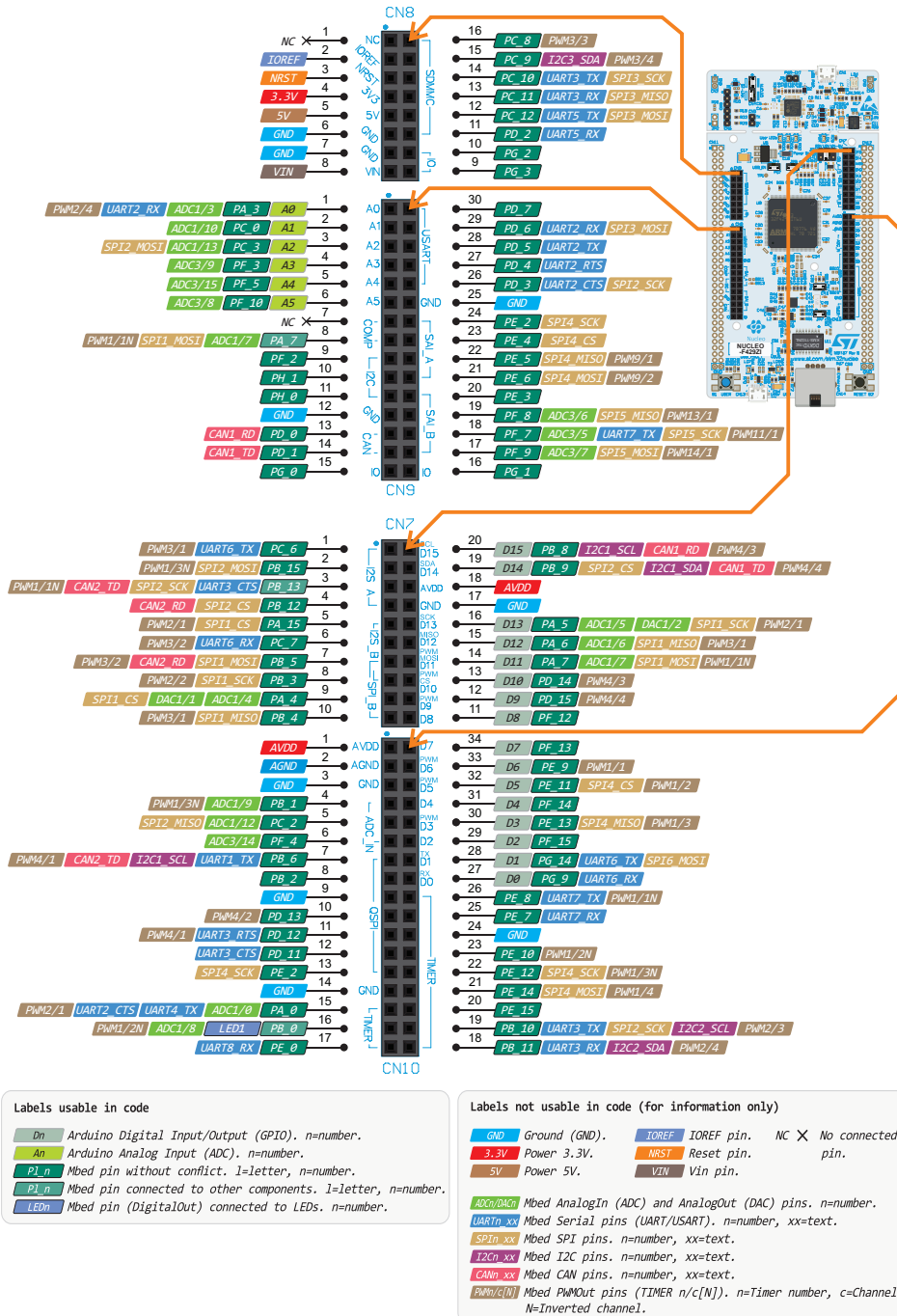


Figure 1.23 ST Zio connectors of the NUCLEO-F429ZI board.

Figure 1.23 shows how different elements of the STM32F429ZIT6U microcontroller are mapped to the Zio and Arduino-compatible headers of the NUCLEO-F429ZI board. Some other elements are mapped to the CN11 and CN12 headers of the NUCLEO-F429ZI board, as will be discussed in upcoming chapters. Further information on these headers is available from [17].

In this chapter, buttons were connected to the NUCLEO board using pins D2 to D7. From Figure 1.23, it can be seen that those digital inputs can also be referred to as PF_15, PE_13, P_14, PE_11, PE_9, and PF_13, respectively. Throughout this book, many pins of the ST Zio connectors will be used, and they will be referred to in the code using the names shown in Figure 1.23.



WARNING: Keil Studio Cloud translates the C/C++ language code into assembly code while considering the available resources of the target board. For this reason, the reader should be very careful to use only pin names that are shown in Figure 1.23.

From the above discussion, it is possible to derive the hierarchy that is represented in Figure 1.24. It should be noted that a given Arm processor can be used by different microcontroller manufacturers, and a given microcontroller can be used in different development boards or embedded systems.

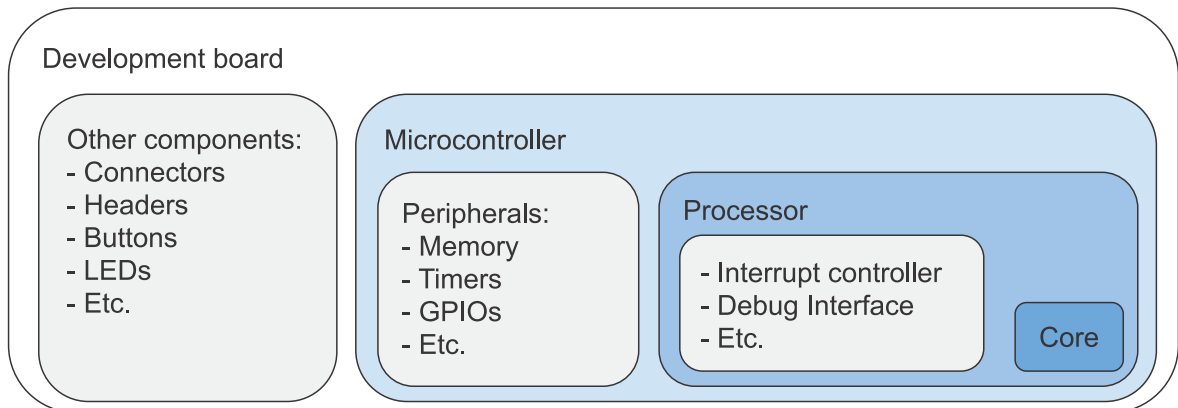


Figure 1.24 Hierarchy of different elements introduced in this chapter.



NOTE: A microcontroller may have one or many processors, while a processor may have one or many cores. A microprocessor consists of the processor, named in this context as the Central Processing Unit (CPU), and uses an external bus to interface with memory and peripherals.

Proposed Exercise

1. How can the code of Example 1.5 be modified in order to use the alternative names of D2 to D7?

Answer to the Exercise

1. It can be achieved by means of the changes shown in Table 1.12.

Table 1.12 Proposed modifications in the code in order to use the alternative names of D2 to D7.

Lines in Code 1.5	New code to be used
2 DigitalIn gasDetector (D2);	2 DigitalIn gasDetector (PF_15);
3 DigitalIn overTempDetector (D3);	3 DigitalIn overTempDetector (PE_13);
4 DigitalIn aButton (D4);	4 DigitalIn aButton (PF_14);
5 DigitalIn bButton (D5);	5 DigitalIn bButton (PE_11);
6 DigitalIn cButton (D6);	6 DigitalIn cButton (PE_9);
7 DigitalIn dButton (D7);	7 DigitalIn dButton (PF_13);

1.4 Case Study

1.4.1 Smart Door Locks

In this chapter, we implemented the functionality to turn on an alarm based on the activation of sensors that were represented by means of buttons. A code was also implemented. A brief view of a commercial “smart door lock” built with Mbed and containing some of these features can be found in [18]. A representation of the system is shown in Figure 1.25.

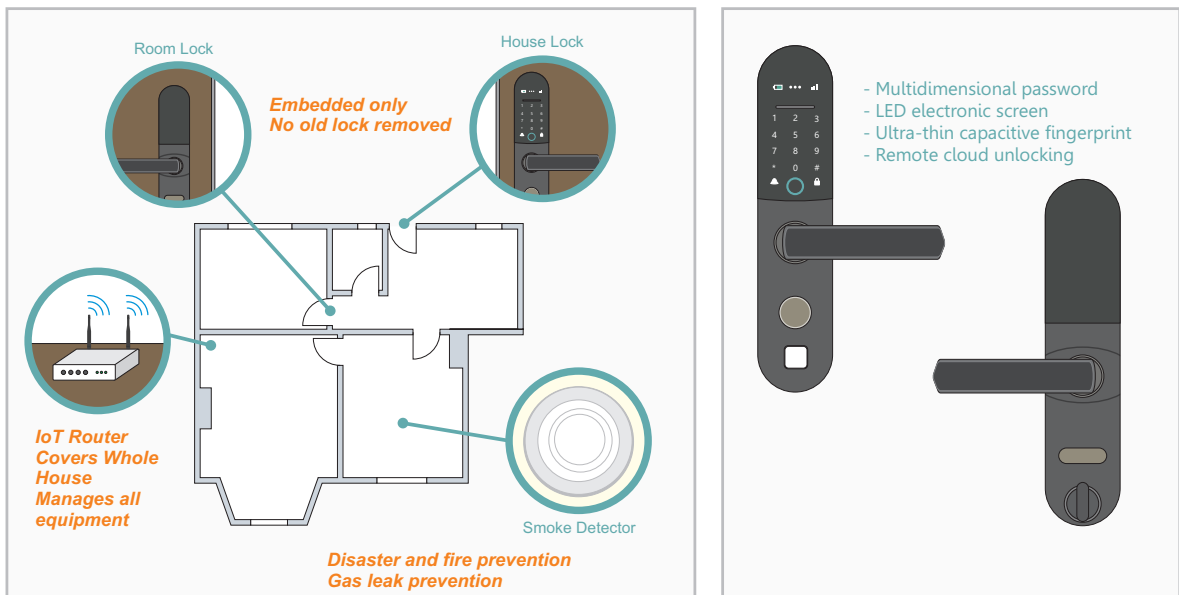


Figure 1.25 “Smart door locks” built with Mbed contains elements introduced in this chapter.

In Figure 1.25 it can be seen that the “smart door lock” has a smoke detector (similar to the gas detector mentioned in this chapter) and a locking system that employs passwords. The system was developed using Mbed OS and is based on a Cortex-M4 core. It is important to highlight that the STM32 microcontroller of the NUCLEO board is also based on a Cortex-M4 core.

Proposed Exercises

1. Are there buttons in Figure 1.25? Where are they located?
2. How is the password entered? Is it like in Example 1.4?

Answers to the Exercises

1. A keypad to enter the password can be seen in Figure 1.25, but there are no switches under the numbers. Instead, ultra-thin capacitive fingerprints are used.
2. The password is entered by pressing the numbers one after the other (not simultaneously like in Example 1.4). Later in this book, it will be explained how to implement a sequential reading of a matrix keypad.

References

- [1] "NUCLEO-F429ZI - STMicroelectronics". Accessed July 9, 2021.
<https://os.mbed.com/platforms/ST-Nucleo-F429ZI/>
- [2] "Introduction - Mbed OS 6 | Mbed OS 6 documentation". Accessed July 9, 2021.
<https://os.mbed.com/docs/mbed-os/v6.12/introduction/index.html>
- [3] "Arm Keil | Cloud-based Development Tools for IoT, ML and Embedded". Accessed July 9, 2021.
<https://www.keil.arm.com/>
- [4] "Log In | Mbed". Accessed July 9, 2021.
<https://os.mbed.com/account/login/>
- [5] "Keil Studio". Accessed July 9, 2021.
<https://studio.keil.arm.com/>
- [6] "Documentation - Arm Developer". Accessed July 9, 2021.
<https://developer.arm.com/documentation>
- [7] "Arm Mbed Studio". Accessed July 9, 2021.
<https://os.mbed.com/studio/>
- [8] "STM32CubeIDE - Integrated Development Environment". Accessed July 9, 2021.
<https://www.st.com/en/development-tools/stm32cubeide.html>
- [9] "STM32F429ZI - High-performance advanced line, Arm Cortex-M4". Accessed July 9, 2021.
<https://www.st.com/en/microcontrollers-microprocessors/stm32f429zi.html>

- [10] "GitHub - armBookCodeExamples/Directory". Accessed July 9, 2021.
<https://github.com/armBookCodeExamples/Directory/>
- [11] "cplusplus.com - The C++ Resources Network". Accessed July 9, 2021.
<https://www.cplusplus.com/>
- [12] "Indentation style - Wikipedia". Accessed July 9, 2021.
https://en.wikipedia.org/wiki/Indentation_style
- [13] "Home - STMicroelectronics". Accessed July 9, 2021.
<https://www.st.com/>
- [14] "Artificial Intelligence Enhanced Computing - Arm". Accessed July 9, 2021.
<https://www.arm.com/>
- [15] "Microprocessors Cores and Technology - Arm". Accessed July 9, 2021.
<https://www.arm.com/products/silicon-ip-cpu>
- [16] "ARM Cortex-M7: Bringing High Performance to the Cortex-M Processor Series. Accessed July 9, 2021.
http://www.armtechforum.com.cn/2014/bj/B-1_BringingHighPerformancetotheCortex-MProcessorSeries.pdf
- [17] "NUCLEO-F429ZI | Mbed". Accessed July 9, 2021.
<https://os.mbed.com/platforms/ST-Nucleo-F429ZI/#zio-and-arduino-compatible-headers>
- [18] "Smart door locks | Mbed". Accessed July 9, 2021.
<https://os.mbed.com/built-with-mbed/smart-door-locks/>